RESQORA

Technical Architecture, Data Flow and UI Guide

This document explains the technologies, backend boundaries, database model, request flows, media handling, security model, deployment path, and the main mobile/public user interfaces used by the current RESQORA implementation.

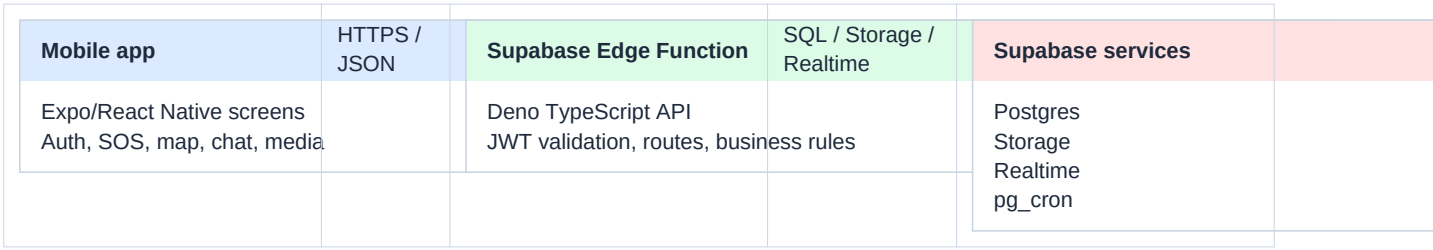
Area	Implemented technology
Mobile client	Expo SDK 57, React Native 0.86, React 19, Expo Router
Production API	Supabase Edge Function written in TypeScript/Deno
Database	Supabase Postgres
Files	Private Supabase Storage bucket with signed URLs
Live updates	Supabase Realtime plus polling fallbacks
Maps	OpenStreetMap, Leaflet, WebView, Nominatim
Distribution	EAS Build APK and EAS Update OTA

Version context: mobile runtime 2.0.2, Android package com.resqora.app, production API at the Supabase Functions URL.

1. System architecture

The production mobile path does not depend on the old laptop/SQLite server. The legacy server folder remains as reference code, while the installed app uses HTTPS to reach the deployed Supabase Edge Function.

Layer	Responsibility	Main implementation
Mobile UI	Screens, navigation, permissions, user actions	Expo Router, React Native JSX
Mobile services	Auth token, API calls, media picker, formatting	app/services/api.js and photo.js
API boundary	Authentication, validation, business rules, signed URLs	supabase/functions/api/index.ts
Data layer	Users, SOS sessions, responders, contacts, timers, chat	Supabase Postgres
Media layer	Private photos/videos and time-limited access	Supabase Storage: sos-media
Live layer	Foreground updates and recovery after app suspension	Supabase Realtime + polling
Public layer	Read-only browser SOS viewer	GitHub Pages docs/index.html



The public viewer follows a separate safe boundary: browser → public token route → read-only tracking JSON. It never receives the service-role key and has no write controls.

2. Technology stack by app segment

Segment	Technology terms	Why it is used
Navigation	Expo Router, file-based routes	Screen navigation and auth redirects
UI	React Native, StyleSheet, shared UI primitives, Inter fonts	Cross-platform native interface and consistent design tokens
Authentication	Supabase Auth plus custom signed RESQORA JWT	Identity plus protected API route authorization
Networking	fetch, JSON, AsyncStorage	HTTPS API calls and device-only session persistence
Location	expo-location, GPS, Haversine formula	Current position, proximity filtering, responder distance
Maps	react-native-webview, Leaflet, OpenStreetMap tiles	Map display without Google Maps billing/API key
Media input	expo-image-picker, expo-video, FileSystem legacy reader	Photo/video capture, selection, validation, playback
Sharing	expo-sharing/clipboard/SMS/linking patterns	Copy links, share sheet, native SMS, directions
Realtime	Supabase Realtime Postgres changes	Foreground SOS updates and chat publication
Scheduled work	Supabase pg_cron	Closed-app safety-timer expiry
Build/update	EAS Build, EAS Update, runtimeVersion 2.0.2	Signed APK distribution and JavaScript OTA updates

Important distinction: expo-video is a native module, so video support requires a new APK when the native dependency is introduced. JavaScript-only fixes such as chat UI/API handling can use OTA when the runtime is compatible.

3. Database and storage model

Supabase Postgres is the production database. The Edge Function uses the service-role key server-side, while Row Level Security remains enabled to protect against accidental direct access through public keys.

Table	Important columns	Relationship / purpose
users	id, name, phone, password, push_token, latitude, longitude, last_seen	Account, location presence, notification target
sos_sessions	victim_id, victim_lat/lon, message, image_url, image_mime_type, status, share_token, timestamps	One emergency session
sos_responders	sos_id, responder_id, status, responded_at, arrived_at	Many-to-many SOS participants and statuses
emergency_contacts	user_id, name, phone	Saved contacts, maximum five per user
safety_timers	user_id, note, status, expires_at, triggered_sos_id	Persistent safety timer and generated SOS link
sos_messages	sos_id, sender_id, body, created_at	Active SOS group chat; maximum 500 characters

Storage bucket: sos-media

Private bucket. Images are limited to 5 MB. Videos are limited to 20 MB and 30 seconds in the mobile picker. Supported video MIME types are video/mp4, video/quicktime, and video/3gpp. The API uploads with the service-role key and returns signed URLs valid for a limited time; the app never receives that key.

Database protections include foreign keys, uniqueness constraints, enum-like status checks, coordinate/range checks, message-length checks, indexes, RLS, and Realtime publication for SOS sessions and chat messages.

4. Main application flows

The following flows show the sequence of UI action, API processing, database mutation, and live updates.

4.1 Authentication flow

Register/Login UI	Supabase Auth	Edge API
User enters identity and password.	Creates or restores Auth session.	Custom API token is issued and cached locally.

The app stores the device session/token in AsyncStorage only for persistence. Supabase service credentials are never bundled into the app.

4.2 SOS creation flow

SOS trigger screen

Hold 3 seconds → 5-second countdown → editable message/media.

↓

Location service

expo-location obtains high-accuracy latitude/longitude.

↓

POST /sos/trigger

Edge Function validates coordinates, message, radius, media and duplicate-SOS state.

↓

Postgres + Storage

Creates sos_sessions, uploads optional media, creates nearby sos_responders.

↓

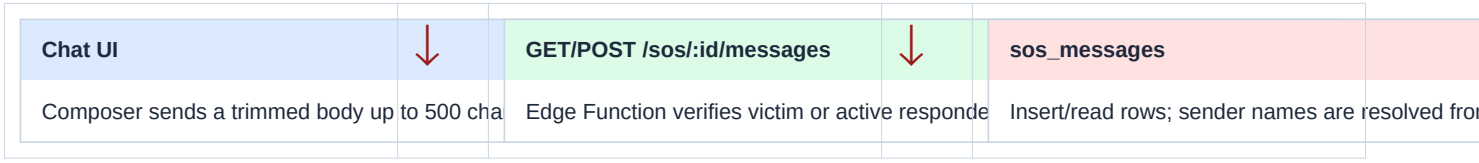
Active SOS UI

Victim sees map, responders, counts, media, link and group chat.

4.3 Responder flow

Step	What happens
1. Discovery	Responder app updates its location. /sos/incoming finds active SOS sessions inside range.
2. Alert	In-app overlay, sound/vibration, victim message, location, distance and media appear.
3. Join	Responder posts /sos/:id/respond. The participant row becomes responding.
4. Navigation	Map, route, victim location updates and responder distance are displayed.
5. Status	Responder chooses arrived or withdraws; status is persisted server-side.
6. Close	Victim ends SOS or the session ends; responder screen navigates to resolved state.

4.4 Group-chat flow



The mobile chat refreshes every five seconds. Realtime publication is enabled for future instant delivery. The public viewer receives only read-only message data through the unguessable token route.

4.5 Media flow

Stage	Implementation
Pick	expo-image-picker chooses image, records video, or selects video.
Validate	Client checks duration, file size, MIME type and supported extension.
Encode	The current JSON API sends a data URI; video reads use expo-file-system/legacy.
Store	Edge Function decodes bytes and uploads to private sos-media Storage.
Expose	Database stores path/type/name; API signs a temporary URL.
Render	Photo component renders image or expo-video player; browser uses img/video controls.

4.6 Safety-timer flow

SafetyTimer UI

Preset/custom duration, note, check-in, extend, cancel.

↓

POST /timer/start

Stores absolute expires_at in safety_timers.

↓

App foreground

Countdown restores from server timestamp; at zero calls /timer/expire.

↓

Closed app

pg_cron checks expired running rows every minute.

↓

Automatic SOS

Timer becomes expired, SOS session is inserted, nearby responders are added.

Atomic status transitions prevent the client and scheduled job from creating duplicate SOS sessions.

4.7 Public-link flow

Step	Technical operation
Create	Active SOS receives a random share_token and GitHub Pages URL.
Open	Browser loads docs/index.html with ?token=...; no login is required.
Poll	Viewer calls /t/:token/data every ten seconds with cache disabled.
Resolve	Edge Function validates token, signs media, returns map/responders/status/chat.
Display	Viewer renders Leaflet map, reverse-geocoded address, media, read-only chat and status.
End	When status is ended, viewer shows the ended state and stops treating it as live.

5. UI and screen map

These wireframes describe the current information hierarchy. They are documentation diagrams, not replacement screenshots; the actual UI is implemented in the linked React Native screens.

HOME / YOU ARE SAFE
Header RESQORA • profile/status
Location Current GPS and connection state
Nearby help Police, hospitals, hotlines, directions
Primary action Emergency SOS
Secondary Safety timer • history • profile

SOS TRIGGER
Safety gate Hold button 3 seconds
Countdown Going out in 5… Send now / cancel
Message Editable emergency text
Media Add photo or video
Preview What responders receive

ACTIVE SOS
Banner SOS live — location shared
Map Victim + responders + route
Stats Elapsed • alerted • coming
Shared Message + media + replace
Chat Private participant group chat
Actions Share link • text contacts • end

RESPONDER
Alert Victim, message, distance, media
Map Route to victim
Status On my way • arrived • withdraw
Chat Participant group chat
Action Open directions

HISTORY
List SOS events grouped by day
Item Status, time, duration, message
Media Photo viewer or video player
People Responder names/count
Action Reopen active SOS

PUBLIC VIEWER
Banner Live or ended
Map Victim + responders
Stats Elapsed • alerted • coming
Shared Message, address, media download
Chat Read-only messages

6. API route groups

Route group	Examples	Purpose
Auth	/auth/register, /auth/login, /auth/password	Account and password lifecycle
User	/user/me, /user/location, /user/push-token	Profile, location and push registration
SOS	/sos/trigger, /sos/active, /sos/:id	Create/read/end emergency sessions
Responder	/sos/:id/respond, /arrived, /withdraw, /sos/incoming	Join and update response state
Media	/sos/:id/image	Attach or replace private SOS media
Chat	/sos/:id/messages	Read/write active participant messages
Contacts	/contacts, /contacts/:id	Manage saved emergency contacts
Timers	/timer/start, /expire, /checkin, /extend, /cancel	Persistent safety timer lifecycle
Public	/t/:token/data	Unauthenticated read-only tracking JSON
Nearby	/nearby/help	OpenStreetMap emergency places and hotlines

Protected routes require the custom RESQORA Bearer token. The public tracking route requires only the unguessable share token and exposes intentionally limited read-only data.

7. Security and privacy boundaries

Protected	How
Service credentials	Supabase service-role key exists only in Edge Function environment.
Database	RLS enabled; API owns controlled access through server-side queries.
Chat	API verifies victim or active responder membership for every read/write.
Media	Private Storage bucket; signed URLs instead of public object paths.
Public link	Token is unguessable; page is read-only and has no write API.
Transport	Production mobile API is HTTPS and Android cleartext traffic is disabled.
Input	Coordinates, radius, message lengths, MIME types and file sizes are validated.

8. Deployment and update flow

Developer source

GitHub repository: Eklavya-Yadav-1711/RESQURA

↓

Supabase deployment

Migrations → Postgres/Storage; Edge Function deploy → production API.

↓

Web deployment

docs/index.html → GitHub Pages public viewer.

↓

Mobile distribution

EAS Build creates signed APK; EAS Update publishes compatible JavaScript changes.

↓

Installed phone

Runtime 2.0.2 app loads OTA update on preview channel when compatible.

Native-module rule: JavaScript changes can generally use OTA, but changes requiring native modules or native configuration require a new APK. The video feature required expo-video and therefore required the final APK build.

9. Source map

Source area	Important files
Mobile routes	app/app/sos-trigger.jsx, sos-active.jsx, responder-nav.jsx, (tabs)/history.jsx
Mobile services	app/services/api.js, photo.js, format.js, share.js
Shared UI	app/components/ui.jsx, Photo.jsx, SOSChat.jsx, LiveMap.jsx
Backend	supabase/functions/api/index.ts
Database	supabase/migrations/20261020_resqora_initial.sql and later migration files
Public UI	docs/index.html
Configuration	app/app.json, app/eas.json, app/.npmrc